Übungsblatt 2: Speicherhierarchie und Caches

Mikrocomputertechnik 3 · Papieraufgaben und optionaler Simulatorvergleich

Musterlösung

Ziel und Arbeitsweise

Sie erklären Lokalität, rechnen die mittlere Speicherzugriffszeit aus und verfolgen Cache-Zustände. Alle Pflichtaufgaben sind ohne Rechner und ohne Simulator lösbar.

Bearbeitungszeit: etwa 60–75 Minuten. **Orientierungspunkte:** 32 Punkte. Für die vier Schulstunden sind insgesamt 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 Minuten), Aufgaben 2–3 (15 Minuten), Aufgaben 4–5 (15 Minuten) und Aufgabe 6 (15 Minuten). Gemeinsame Besprechungen erfolgen in den Vorlesungsphasen; die längere Zeitangabe gilt für selbstständiges Bearbeiten. Begründen Sie Ihre Antworten; zu Rechnungen gehören Formel, Rechenweg und Einheit. Hexadezimale Adressen tragen das Präfix `0x`. Eine Adresse bezeichnet ein Byte.

Wir verwenden ausdrücklich vereinfachte Lehrmodelle: ein Prozessor, normaler cachefähiger Speicher, abgeschlossene Anfragen, kein Vorabladen und keine zusätzlichen Zugriffe. „Kalt“ bedeutet, dass alle Valid-Bits zunächst null sind. Valid ist das Gültigkeitsbit, Dirty das Änderungsbit einer Write-Back-Line. Zähler und Zeiten der Modelle sind keine Messungen eines übersetzten C-Programms.

Aufgabe 1: Lokalität begründen (4 Punkte)

Die beiden Folgen beginnen jeweils unabhängig mit einem kalten Cache und 64-Byte-Lines. Es findet keine Verdrängung statt.

- A: `0x1000, 0x1000, 0x1000`
- B: `0x1000, 0x1004, 0x1008`

1. Ordnen Sie zeitliche und räumliche Lokalität zu. Begründen Sie die Zuordnung. (2 P)
2. Bestimmen Sie für jede Folge Hits und Misses. Erklären Sie, warum eine verstreute verkettete Liste mit demselben Elementumfang schlechtere räumliche Lokalität haben kann, aber nicht muss. (2 P)

Musterlösung 1

A zeigt zeitliche Lokalität: Dieselbe Adresse wird erneut gelesen. B zeigt räumliche Lokalität: Benachbarte Wörter werden gelesen. Alle Adressen gehören zur Line `0x1000` bis `0x103F`. In beiden unabhängigen Folgen entsteht daher zuerst ein Miss, danach zwei Hits. Das gilt unter der Annahme, dass die gültige Line erhalten bleibt.

Ein Array speichert seine Elemente zusammenhängend. Bei einer verstreuten Liste kann der Zeiger auf einen Knoten in einer anderen Line führen; die mitgeladenen Nachbarn helfen dann wenig. Zusammenhängend abgelegte Listenknoten können dagegen ebenfalls gute räumliche Lokalität haben. Der Datentyp allein legt die Quote nicht fest. Bewertung: je 1 P für die zwei Zuordnungen, 1 P für die Bilanzen, 1 P für die begründete Layout-Aussage.

Aufgabe 2: AMAT auf zwei Wegen (5 Punkte)

Hit Time $t_H = 1$ Takt, **zusätzliche** Miss Penalty $p = 100$ Takte. AMAT bedeutet Average Memory Access Time, mittlere Speicherzugriffszeit.

1. Eine Folge mit 100 Anfragen hat 95 Hits und 5 Misses. Berechnen Sie die Miss Rate und AMAT. Kontrollieren Sie das Ergebnis durch Aufsummieren aller Einzelzeiten. (3 P)
2. Berechnen Sie AMAT bei 80 % Hit Rate. (1 P)
3. Ist das Verhältnis der beiden AMAT-Werte bereits der Beschleunigungsfaktor eines ganzen Programms? Begründen Sie. (1 P)

Musterlösung 2

Die Miss Rate m ist der Anteil der Misses:

$$m = \frac{5}{100} = 0{,}05, \qquad \text{AMAT} = t_H + mp = 1 + 0{,}05 \cdot 100 = 6 \text{ Takte.}$$

Jeder Miss kostet insgesamt $1 + 100 = 101$ Takte. Kontrolle:

$$\frac{95 \cdot 1 + 5 \cdot 101}{100} = \frac{600}{100} = 6 \text{ Takte.}$$

Bei 80 % Hit Rate ist $m = 1 - 0{,}80 = 0{,}20$ und $\text{AMAT} = 1 + 0{,}20 \cdot 100 = 21$ Takte. Die Prüfung kostet auch beim Miss einen Takt; daher sind 5 beziehungsweise 20 Takte keine korrekten Ergebnisse.

Das Verhältnis $21/6 = 3{,}5$ vergleicht die mittleren Zugriffszeiten des Modells. Für eine Programmlaufzeit fehlen unter anderem Anfrageanzahl, übrige Rechenarbeit und Überlappung. Bewertung: Miss Rate 1 P, Formel und Ergebnis 1 P, Kontrollrechnung 1 P, zweite AMAT 1 P, begründete Abgrenzung 1 P.

Aufgabe 3: Adressfelder und Blockgrenzen (6 Punkte)

Referenzmodell aus der Vorlesung: 32-Bit-Byteadressen, $C = 32$ KiB Datenkapazität, $B = 64$ Byte je Line. Ein KiB hat 1024 Byte. Tags und Zustandsbits zählen nicht zu C.

1. Bestimmen Sie Anzahl der Sets sowie Bitbreiten von Tag, Index und Offset für Direct Mapped und für 4-Way. Erklären Sie, warum der Offset gleich bleibt. (3 P)
2. Zerlegen Sie `0x12345678` für Direct Mapped. Geben Sie Index und Offset dezimal, das Tag hexadezimal an. Kontrollieren Sie durch Rückrechnung. (2 P)
3. Eine andere Anfrage hat die **Dezimaladresse** 1000. Geben Sie bei 64-Byte-Lines Blockbeginn, Blockende und Offset an. (1 P)

Musterlösung 3

Mit W Ways, also Plätzen je Set, gilt

$$S = \frac{C}{BW}, \quad b = \log_2 B, \quad s = \log_2 S, \quad \tau = 32 - b - s.$$

Organisation	Sets S	Tagbits τ	Indexbits s	Offsetbits b
Direct Mapped	512	17	9	6
4-Way	128	19	7	6

Die Blockgröße bleibt 64 Byte; der Offset adressiert weiterhin dieselben Bytepositionen. Vier Ways verteilen dieselben 512 Plätze auf weniger Sets.

Für `0x12345678` ist $o = 56$, $q = \lfloor a/64 \rfloor =$ `0x48D159`, $i = q \bmod 512 = 345$ und $t = \lfloor q/512 \rfloor =$ `0x2468` $= 9320$. Die Rückrechnung lautet

$$a = t \cdot 32768 + i \cdot 64 + o = 9320 \cdot 32768 + 345 \cdot 64 + 56 = 305419896 = \texttt{0x12345678}.$$

Für Dezimaladresse 1000 ist die Blocknummer 15. Blockbeginn $15 \cdot 64 = 960$, Blockende $960 + 63 = 1023$, Offset $1000 - 960 = 40$. Bewertung: je 1 P pro Tabellenzeile, 1 P für unveränderten Offset, 1 P für Adressfelder, 1 P für Rückrechnung, 1 P für Blockgrenzen und Offset.

Aufgabe 4: Zugriffstabelle und Ersetzung (6 Punkte)

4a: Ein Platz oder vier Plätze? (4 Punkte)

Verwenden Sie das Referenzmodell aus Aufgabe 3. Beide Caches beginnen kalt. Alle Anfragen sind Daten-Loads; vier Ways verwenden zunächst freie Plätze.

Schritt	Adresse	DM: Index	DM: Tag	DM: Hit/Miss	4-Way: Hit/Miss
1	`0x1000`				
2	`0x1004`				
3	`0x9000`				
4	`0x1000`				
5	`0x9000`				
6	`0x1004`				

DM steht für Direct Mapped. Füllen Sie die Tabelle aus. Nennen Sie für jeden Cache Hits, Misses und Hit Rate. Begründen Sie, warum die zusätzlichen DM-Misses Konflikt-Misses und keine Kapazitäts-Misses sind.

4b: FIFO oder LRU? (2 Punkte)

Ein getrenntes, zunächst leeres 4-Way-Set erhält die Blockfolge `A, B, C, D, A, E`. Alle Blöcke haben denselben Set-Index.

Geben Sie für FIFO (First In, First Out) und LRU (Least Recently Used) die Reihenfolge „alt nach neu“ nach dem zweiten A sowie den durch E verdrängten Block an. Erklären Sie die Wirkung eines Hits auf beide Regeln.

Musterlösung 4

Schritt	Adresse	DM: Index	DM: Tag	DM	4-Way
1	`0x1000`	64	0	Miss	Miss
2	`0x1004`	64	0	Hit	Hit
3	`0x9000`	64	1	Miss	Miss
4	`0x1000`	64	0	Miss	Hit
5	`0x9000`	64	1	Miss	Hit
6	`0x1004`	64	0	Miss	Hit

`0x1004` gehört zur selben Line wie `0x1000`. Direct Mapped hat 1 Hit, 5 Misses und Hit Rate $1/6 \approx 16{,}67\,\%$. Zwei Misses sind erste Blockanforderungen, drei weitere entstehen durch gegenseitige Verdrängung.

Der 4-Way-Cache hat 4 Hits, 2 Misses und Hit Rate $4/6 \approx 66{,}67\,\%$. Beide Blöcke treffen dort ebenfalls Set 64, mit Tags 0 und 4. Sie belegen verschiedene Ways. Schon zwei Plätze würden für diese Folge genügen. Die zusätzlichen DM-Misses entstehen trotz ausreichend großer Gesamtkapazität: Ein vollassoziativer Cache gleicher Kapazität könnte beide Blöcke halten. Bewertung 4a: Index/Tag 1 P, Trefferfolgen 1 P, Bilanzen und Quoten 1 P, Konfliktbegründung 1 P.

FIFO behält nach dem Hit auf A die Ladereihenfolge A, B, C, D und verdrängt A. LRU aktualisiert die letzte Nutzung zu B, C, D, A und verdrängt B. Ein Hit verjüngt einen Block nur in der LRU-Nutzungsordnung. Bewertung 4b: je 1 P für vollständige Reihenfolge, Opfer und Begründung pro Regel.

Aufgabe 5: Schreiben und alte Blockadresse (5 Punkte)

Direct Mapped wie Aufgabe 3, Write-Back und Write-Allocate, keine Puffer. Zunächst ist die Line `0x1000` gültig und sauber (Dirty = 0). Das betrachtete Wort hat im Cache und in der nächsten Ebene den Wert 0. `0x9000` ist nicht im Cache; dort enthält die nächste Ebene den Wert 9.

Verfolgen Sie: `Store 5 nach 0x1000; Load 0x1000; Store 7 nach 0x1000; Load 0x9000`. Nur diese vier Anfragen werden gezählt.

1. Füllen Sie Zustand und eventuelles Writeback nach jedem Schritt aus. (3 P)

Schritt	Cache-Blockbasis	Wortwert	Dirty	Wert unten bei `0x1000`	Writeback-Adresse
Store 5					
Load					
Store 7					
Load `0x9000`					

2. Warum muss der letzte Schritt gegebenenfalls an die alte Blockadresse zurückschreiben? (1 P)
3. Was wäre bei einem Store-Miss mit No-Write-Allocate anders? Ist diese Regel durch Write-Back schon festgelegt? (1 P)

Musterlösung 5

Schritt	Blockbasis	Wortwert	Dirty	Wert unten bei `0x1000`	Writeback
Store 5	`0x1000`	5	1	0	keiner
Load	`0x1000`	5	1	0	keiner
Store 7	`0x1000`	7	1	0	keiner
Load `0x9000`	`0x9000`	9	0	7	nach `0x1000`

Die ersten drei Anfragen sind Hits. Lesen ändert Dirty nicht. Der vierte Zugriff ist ein Miss auf denselben Index 64 und verdrängt die veränderte Line. Die alte Blockbasis ist aus altem Tag und Index zu bilden:

$$a_{\text{alt}} = (t_{\text{alt}} \cdot S + i)B = (0 \cdot 512 + 64) \cdot 64 = \texttt{0x1000}.$$

Das Zurückschreiben erhält den aktuellen Wert 7 am richtigen Ort. Ein Writeback nach `0x9000` würde den fremden Block überschreiben. Danach wird der angefragte Block geladen und ist sauber. Die gültige Line bleibt bei allen dargestellten Zuständen Valid = 1.

No-Write-Allocate würde bei einem Store-Miss den Store weitergeben, ohne eine neue Cache-Line anzulegen. Weitergabe und Zuteilen sind getrennte Entscheidungen; Write-Back legt die Store-Miss-Regel nicht allein fest. Bewertung: Stores 1 P, Load-Hit 1 P, letzter Zustand 1 P, alte Adresse 1 P, Policy-Abgrenzung 1 P.

Aufgabe 6: C-Layout als Papieraufgabe (6 Punkte)

Es gilt `uint32_t a[16][16]`: 16 Zeilen, 16 Spalten, 4 Byte pro Element, Basis `0x1000`. Die Elemente sind initialisiert. Betrachten Sie zwei **definierte Modellspuren**, je genau ein Daten-Load pro Element. Andere Zugriffe und Compileroptimierungen gehören nicht zur Zählung.

```
/* A: zeilenweise; sum ist ein uint64_t-Akkumulator mit Startwert 0. */
for (unsigned r = 0; r < 16; ++r)
    for (unsigned c = 0; c < 16; ++c)
        sum += a[r][c];

/* B: unabhängig davon spaltenweise, sum zuvor wieder 0. */
```

```
for (unsigned c = 0; c < 16; ++c)
    for (unsigned r = 0; r < 16; ++r)
        sum += a[r][c];
```

`uint32_t` und `uint64_t` sind vorzeichenlose Ganzzahltypen mit 32 beziehungsweise 64 Bit; `r` und `c` sind Zeilen- und Spaltenindex.

1. Leiten Sie die Elementadresse her und geben Sie die ersten vier Adressen beider Spuren an. (2 P)
2. Kleines Modell: 512 Byte Datenkapazität, Direct Mapped, 64-Byte-Lines, vor jeder Variante kalt. Leiten Sie den Set-Index der Zeile r her und zählen Sie Hits und Misses für beide vollständigen Spuren. (3 P)
3. Ändern Sie nur die Datenkapazität auf 1024 Byte. Was ändert sich an der spaltenweisen Bilanz? Begründen Sie. (1 P)

Musterlösung 6

C speichert zeilenweise. Vor Zeile r liegen $16r$ Elemente, davor in der Zeile weitere c. Somit

$$a_{r,c} = \texttt{0x1000} + 4(16r + c).$$

Spur A beginnt `0x1000, 0x1004, 0x1008, 0x100C`. Spur B beginnt `0x1000, 0x1040, 0x1080, 0x10C0`.

Im kleinen Modell gibt es $512/64 = 8$ Sets. Eine Arrayzeile ist $16 \cdot 4 = 64$ Byte groß und beginnt auf einer Blockgrenze. Ihre Blocknummer ist $64+r$, also ihr Set-Index $(64+r) \bmod 8 = r \bmod 8$.

Zeilenweise folgen auf den ersten Load einer Zeile fünfzehn Hits: 16 Misses und 240 Hits, Hit Rate 93,75 %. Spaltenweise ersetzen Zeilen 8 bis 15 die Zeilen 0 bis 7 in denselben Sets. In der nächsten Spalte fehlen die ersten acht Zeilen wieder; sie ersetzen die zweiten acht, die anschließend ebenfalls fehlen. Daher 256 Misses und 0 Hits. Die 16 ersten Blockanforderungen sind Kaltstart-Misses. Die späteren Misses sind gegenüber einem vollassoziativen Acht-Plätze-Cache mit LRU ebenfalls nicht vermeidbar; die aktive Menge ist zu groß.

Mit 1024 Byte gibt es 16 Sets. Jede Zeile hat einen eigenen Platz und bleibt bis zur Wiederverwendung erhalten. Spaltenweise lädt die erste Spalte alle 16 Lines; die folgenden 15 Spalten ergeben 240 Hits. Beide Varianten haben nun 16 Misses und 240 Hits. Ein großer Adresssprung allein beweist somit keine bestimmte Miss Rate oder Programmlaufzeit.

Bewertung: Adressformel 1 P, Anfangsspuren 1 P, Set-Index 1 P, je eine begründete Bilanz 1 P, größerer Cache 1 P.

Optional: Simulatorvergleich (ohne Punkte)

Wenn Ripes, ein visueller RISC-V-Simulator, verfügbar ist, vergleichen Sie eine explizite Load-Spur mit der Papierlösung. Dokumentieren Sie Version, Cache-Größe, Blockgröße, Ways, Anfangszustand und welche Datenanfragen tatsächlich gezählt werden. Setzen Sie den Cache vor jedem Vergleich zurück.

Behandeln Sie Trefferzahlen als Analyse der Spur. Das Cache-Modell von Ripes ist kein vollständiges Cache-Latenzmodell und belegt keinen allgemeinen Geschwindigkeitsfaktor. Zusätzliche Datenzugriffe eines Simulatorprogramms müssen gesondert berücksichtigt werden.

Hinweis zur Auswertung: Eine Abweichung zuerst auf zusätzliche Zugriffe, falsche Einstellungen oder einen nicht zurückgesetzten Cache prüfen. Keine bestimmte Simulatorzeit ist als Musterergebnis vorgegeben.